LAPORAN HASIL PRAKTIKUM PRAKTIKUM PEMOGRAMAN BERBASIS OBJEK
JOBSHEET X
POLIMORFISME dan EXCEPTION

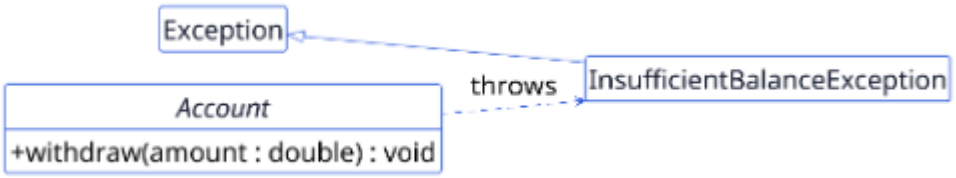


Disusun oleh: Muhammad Toriq Januarsyah
Kelas : TI-2B
NIM : 254107020233
Tanggal : 06 November 2026

PROGRAM STUDI D-IV TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

1. Langkah Kerja - withdraw() Melempar InsufficientBalanceException

1.1 Membaca dan memahami diagram



Gambar 1.1: InsufficientBalanceException subclass Exception, dilempar oleh Account

Cara membaca diagram: InsufficientBalanceException adalah subclass dari Exception. Panah putus-putus berlabel "throws" berarti withdraw() di Account bisa melempar exception itu. Tipe kembalian withdraw() kini void, bukan boolean.

1.2 Buat berkas baru InsufficientBalanceException.java

```java
package jobsheet10;

public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

```

gambar 1.2 : InsufficientBalanceException.java

Kode ini menunjukkan pembuatan kelas eksepsi kustom InsufficientBalanceException yang mewarisi kelas bawaan Exception, berfungsi sebagai penanda spesifik yang akan diinstansiasi ketika terjadi kegagalan transaksi akibat saldo yang tidak mencukupi.

1.3 Ubah withdraw() pada Account.java agar melempar exception ketika canWithdraw() bernilai false

```java
public void withdraw(double amount) throws InsufficientBalanceException {
        if (!canWithdraw(amount)) {
            throw new InsufficientBalanceException(
                accountNumber + ": insufficient balance for a withdrawal of " + amount);
        }
        balance -= amount;
    }
```

Gambar 1.3: Account.java, withdraw melempar InsufficientBalanceException

Modifikasi pada metode withdraw() mendemonstrasikan implementasi klausa throws, di mana program akan melempar objek InsufficientBalanceException secara eksplisit dan memutus alur normal eksekusi apabila kondisi saldo tidak memenuhi regulasi penarikan

1.4 ganti isi Main.java, setiap pemanggilan withdraw() kini berada di dalam try/catch:

```java
package jobsheet10;


public class Main {
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        try {
            savings.withdraw(70000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }

        try {
            savings.withdraw(30000);
            System.out.println("Withdrawal succeeded, new balance: " + savings.getBalance());
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }
    }
}

```

Gambar 1.4 Main.java menguji withdraw yang melempar exception

Pemanggilan instruksi withdraw() kini dibungkus di dalam struktur pelindung try-catch guna menangkap dan menangani eksepsi yang dilempar oleh kelas induk, sehingga program utama mampu mencetak pesan galat secara terstruktur tanpa mengalami penghentian paksa.

- **Checkpoint1:**

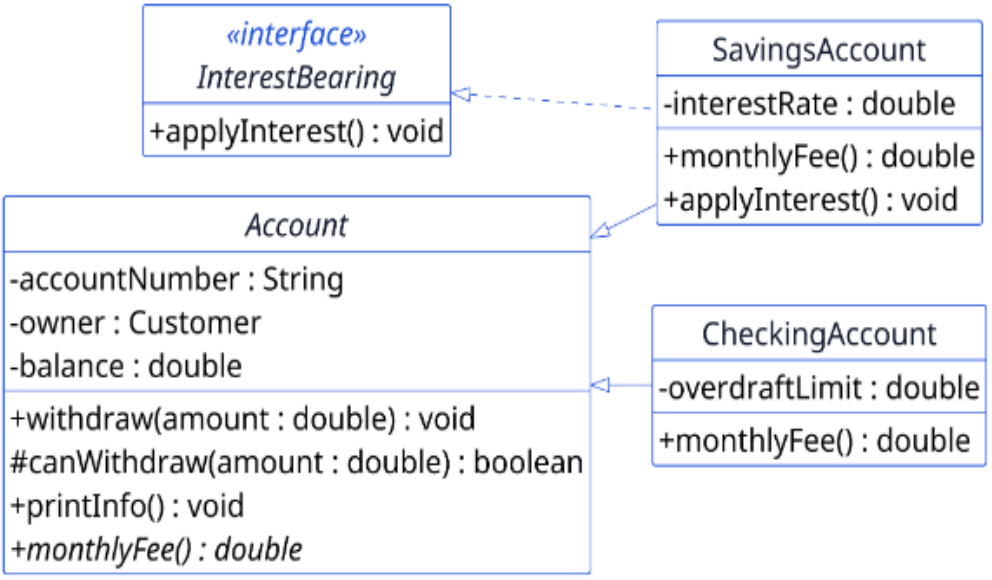
```
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
```

Gambar 1.6: Output Main.java

Hasil eksekusi membuktikan bahwa penarikan pertama yang melampaui batas minimum berhasil memicu pelemparan eksepsi yang kemudian ditangkap oleh blok catch untuk mencetak galat, sedangkan penarikan kedua berjalan pada jalur normal tanpa memicu anomali sistem.

2. Langkah kerja - processMonthEnd(), Polimorfisme dan instanceof

2.1 Membaca dan memahami diagram



Gambar 2.1: .Account sebagai kelas abstrak, SavingsAccount meng-implement interface InterestBear-ing

Account bertindak sebagai kelas induk (superclass) bagi dua kelas turunan di bawahnya, yakni SavingsAccount dan CheckingAccount, di mana kedua kelas turunan mendefinisikan ulang metode monthlyFee(). Diagram ini secara spesifik menampilkan interface InterestBearing yang memiliki metode applyInterest() dan hanya dihubungkan melalui garis putus-putus kepada kelas SavingsAccount. garis ini menginformasikan bahwa kapabilitas fitur bunga murni hanya dimiliki oleh SavingsAccount, sementara CheckingAccount tidak mengimplementasikan antarmuka tersebut.

2.2 Tambahkan method processMonthEnd(): di Bank.java

```java
public void processMonthEnd() {
        for (int i = 0; i < count; i++) {
            Account acc = accounts[i];
            if (acc instanceof InterestBearing bearing) {
                bearing.applyInterest();
                System.out.println(acc.getAccountNumber() + "interest applied, new balance: " + acc.getBalance());
            }
            System.out.println(acc.getAccountNumber() + " monthly fee: " + acc.monthlyFee());
        }
    }
```

Gambar 2.2: Bank.java dengan method processMonthEnd

Penambahan metode processMonthEnd() menerapkan konsep polimorfisme untuk memproses seluruh rekening dalam satu perulangan tunggal, memanfaatkan operator instanceof untuk menyeleksi secara dinamis objek yang memiliki kapabilitas InterestBearing guna pengaplikasian bunga otomatis.

2.3 Tambahkan pemanggilannya di akhir Main.java:

```java
package jobsheet10;


public class Main {
    public static void main(String[] args) {
        Customer customer = new Customer("Rian", "0812-0000-0003");
        SavingsAccount acc1 = new SavingsAccount("A001", customer, 500000, 0.01);
        CheckingAccount acc2 = new CheckingAccount("A002", customer, 200000, 100000);
        SavingsAccount savings = new SavingsAccount("A003", customer, 100000, 0.02);

        try {
            savings.withdraw(70000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }

        try {
            savings.withdraw(30000);
            System.out.println("Withdrawal succeeded, new balance: " + savings.getBalance());
        } catch (InsufficientBalanceException e) {
            System.out.println("Withdrawal failed: " + e.getMessage());
        }

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.processMonthEnd();
    }
}

```

Gambar 2.3: Main.java memanggil processMonthEnd

Instruksi bank.processMonthEnd() ditambahkan untuk mengeksekusi logika pemrosesan akhir bulan secara serentak dan terpusat terhadap seluruh instansi objek rekening yang telah diinjeksi ke dalam wadah penyimpanan kelas Bank.

- **Checkpoint2:**

```
A001interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003interest applied, new balance: 71400.0
A003 monthly fee: 0.0
```

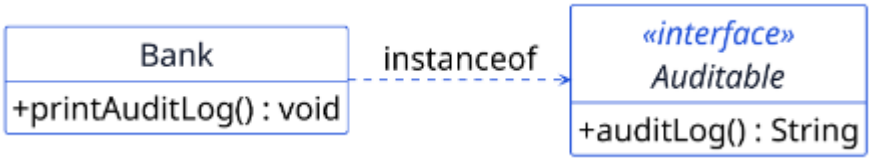
Gambar 2.4: Output Main.java

Keluaran sistem memvalidasi keberhasilan polimorfisme, di mana pemotongan biaya bulanan dieksekusi secara otomatis sesuai kelas aslinya masing-masing, sementara komputasi penambahan bunga hanya diterapkan pada instansi rekening yang terbukti mengimplementasikan antarmuka InterestBearing.

3. Tugas Mandiri.

.

1. Bank memerlukan laporan audit untuk rekening yang Auditable (interface dari topik Kelas Abstrak dan Interface). Tambahkan Bank.printAuditLog(): satu looping atas semua rekening, yang mencetak auditLog() hanya untuk rekening yang Auditable. Diagram berikut hanya sketsa method yang perlu ditambahkan, BUKAN kode jadi, isinya diserahkan sepenuhnya padamu:



Hasilmu benar apabila hanya rekening giro yang tercetak, dan pengecekannya memakai iinstanceof Auditable, bukan nama kelas.

- **Menambah Method printAuditLog() pada class Bank yang menerapkan instanceof pada interface auditable.**

```java
public void printAuditLog() {
        System.out.println("--- Audit Log Report ---");
        for (int i = 0; i < count; i++) {
            Account acc = accounts[i];

            // Memeriksa apakah rekening memiliki kemampuan Auditable
            if (acc instanceof Auditable auditableAcc) {
                System.out.println(auditableAcc.auditLog());
            }
        }
    }
```

Gambar 3.1: Class Auditable dengan method auditLog()

metode printAuditLog() diawali dengan eksekusi struktur perulangan (looping) untuk memindai secara berurutan seluruh objek rekening yang terdaftar di dalam susunan data kelas Bank. Pada setiap iterasi, program menerapkan mekanisme seleksi kondisi dinamis menggunakan operator instanceof untuk memverifikasi secara spesifik apakah entitas rekening yang sedang dievaluasi memiliki kapabilitas atau mengimplementasikan antarmuka Auditable. Apabila hasil verifikasi tersebut terbukti valid, sistem secara otomatis melakukan konversi tipe data terhadap objek terkait menjadi referensi auditableAcc, yang seketika memberikan akses aman bagi program untuk memanggil dan mencetak string luaran dari metode auditLog() tanpa memicu galat struktural pada tipe rekening lain yang tidak didukung.

- **menambahkan printAuditableLog() agar bisa dipanggil di Main.java**



Gambar 3.2: Main.java memanggil printAuditLog();

pada potongan kode di kelas utama (Main) tersebut mendemonstrasikan eksekusi instruksi program secara sekuensial. Instruksi pemanggilan bank.printAuditLog() diletakkan secara terstruktur dan terencana tepat setelah metode pemrosesan akhir bulan (bank.processMonthEnd()) diselesaikan secara utuh.

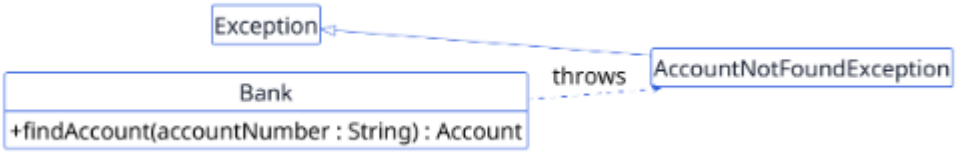
- **Output**

```
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003interest applied, new balance: 71400.0
A003 monthly fee: 0.0
--- Audit Log Report ---
Audit Log - Account: A002, Balance: 200000.0
```

Gambar 3.3: output Main.java

Modifikasi metode printAuditLog() pada kelas Bank berhasil memvalidasi fungsionalitas dari operator instanceof dalam sebuah struktur perulangan. Hasil keluaran program menunjukkan bahwa dari ketiga rekening yang diproses, sistem hanya mencetak informasi jejak audit untuk rekening A002 yang bertipe CheckingAccount. Hal ini membuktikan bahwa seleksi kondisi berbasis interface Auditable beroperasi secara presisi, di mana objek dari kelas SavingsAccount (A001 dan A003) secara otomatis diabaikan karena tidak mengimplementasikan kapabilitas pencatatan audit tersebut.

2. Bank.findAccount() mengembalikan null ketika rekening tidak ditemukan. Pemanggil bisa lupa memeriksa null. Ubah supaya method itu melempar exception AccountNotFoundException. Diagram berikut hanya sketsa struktur dan signature yang berubah, BUKAN kode jadi:



Buktikan lewat Main.java: panggil findAccount() untuk satu nomor rekening yang ada dan satu yang tidak ada, masing-masing di dalam try/catch. Hasilmu benar apabila nomor yang tidak ada menghasilkan pesan dari blok catch, dan program tetap berjalan sampai selesai.

- **Buat file baru bernama AccountNotFoundException.java**

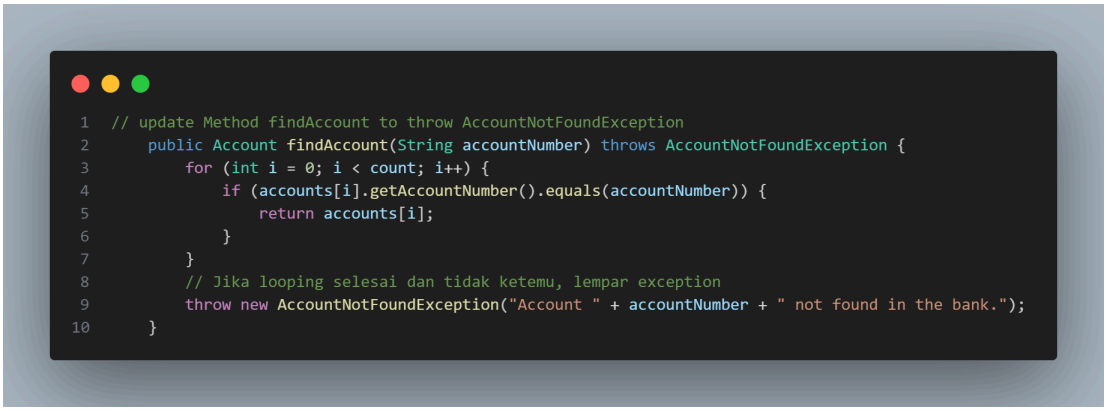
```java
package jobsheet10;

public class AccountNotFoundException extends Exception {
    public AccountNotFoundException(String message) {
        super(message);
    }
}
```

Gambar 4.1: Class AccuntNotFoundExcepton extend Exception

dengan pendefinisian kelas eksepsi kustom AccountNotFoundException yang mewarisi kelas Exception, menjadikannya sebagai checked exception yang mewajibkan penanganan galat secara struktural

- **Modifikasi metode findAccount() pada Bank.java dengan melempar exception**



```java
// update Method findAccount to throw AccountNotFoundException
    public Account findAccount(String accountNumber) throws AccountNotFoundException {
        for (int i = 0; i < count; i++) {
            if (accounts[i].getAccountNumber().equals(accountNumber)) {
                return accounts[i];
            }
        }
        // Jika looping selesai dan tidak ketemu, lempar exception
        throw new AccountNotFoundException("Account " + accountNumber + " not found in the bank.");
    }
```

Gambar 4.2: modifikasi metode findAccount()

modifikasi metode findAccount() di kelas Bank, algoritma pencarian mengeksekusi perulangan untuk mencocokkan parameter nomor rekening dengan data di dalam tempat penyimpanan. Apabila seluruh iterasi pemindaian telah selesai dan entitas yang dicari terbukti tidak eksis, metode ini secara arsitektural tidak lagi mengembalikan nilai referensi kosong (null), melainkan secara proaktif menginstansiasi dan melempar (throw) objek eksepsi kustom tersebut. Mekanisme intersepsi ini secara logis memutus aliran eksekusi pencarian normal dan mendelegasikan tanggung jawab penanganan kegagalan data secara langsung kepada kelas pemanggil, sehingga sistem terlindungi dari potensi galat fatal berantai seperti NullPointerException pada tahap komputasi selanjutnya.

- **Tambahkan kode Uji coba di bagian bawah Main.java**



```java
// kode sebelumnya
        System.out.println("--- Find Account Test ---");

        // 1. Mencari rekening yang ADA
        try {
            Account found = bank.findAccount("A001");
            System.out.println("Account found: " + found.getAccountNumber());
        } catch (AccountNotFoundException e) {
            System.out.println(e.getMessage());
        }

        // 2. Mencari rekening yang TIDAK ADA
        try {
            Account notFound = bank.findAccount("A999");
            System.out.println("Account found: " + notFound.getAccountNumber());
        } catch (AccountNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
```

Gambar 4.3: menambahkan try & catch uji coba berhasil dan gagal pada Main.java

pengujian ini mendemonstrasikan mekanisme perlindungan eksekusi menggunakan struktur try-catch secara berurutan dalam menangani dua skenario validasi data.

- **Output**

```
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003interest applied, new balance: 71400.0
A003 monthly fee: 0.0
--- Audit Log Report ---
Audit Log - Account: A002, Balance: 200000.0
--- Find Account Test ---
Account found: A001
Account A999 not found in the bank.
```

Gambar 4.4: Output Main.java

Modifikasi metode findAccount() pada kelas Bank dengan mengimplementasikan pelemparan AccountNotFoundException terbukti meningkatkan ketahanan sistem dalam menangani kasus pencarian data yang nihil. Hasil keluaran memvalidasi bahwa ketika rekening A001 dicari, metode berhasil mengembalikan objek secara normal. Sebaliknya, ketika rekening fiktif A999 dicari, metode secara eksplisit memicu eksepsi yang kemudian ditangkap oleh struktur pelindung catch pada kelas Main, sehingga program mampu mencetak pesan galat yang informatif tanpa mengalami penghentian eksekusi secara paksa..

4. Kesimpulan

- Dari praktikum ini, saya telah berhasil menerapkan dua konsep utama, yaitu penanganan kesalahan dan sifat banyak bentuk pada objek. Pertama, terkait penanganan kesalahan, sistem yang sebelumnya hanya diam saat terjadi kegagalan (seperti saldo kurang atau data tidak ditemukan) kini diubah agar secara aktif melempar pesan galat. Hal ini terbukti membuat program lebih aman karena memaksa kita untuk menyiapkan blok pelindung, sehingga aplikasi terhindar dari kerusakan fatal atau mati mendadak. Kedua, saya berhasil memproses berbagai jenis rekening secara serentak hanya dengan menggunakan satu perulangan. Dengan memanfaatkan fitur pengecekan kemampuan pada antarmuka, program secara pintar dapat mengenali dan menjalankan fungsi khusus, seperti penambahan bunga dan pencetakan jejak audit hanya pada rekening yang memang mendukung fitur tersebut. Secara keseluruhan, penerapan kedua konsep ini membuat struktur kode menjadi lebih ringkas, sangat aman dari kemungkinan macet, dan lebih mudah untuk ditambah fitur baru di masa depan.

5. Lampiran

https://github.com/ToriqJanuarsyah/Praktikum_PBO.git